Assignment 3

Report

Home PC (Local)	**Linux Lab (Remote)**
Processor: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz	Processor: Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
Physical cores: 4 (Total logical CPUs: 8)	Physical cores: 4 (Total logical CPUs: 4)
Operating System: Ubuntu 24.04.3 LTS	Operating System: Ubuntu 20.04.6 LTS
GCC version: 13.3.0	GCC version: 9.4.0

Name: Pavlos Tomazos **ID:** 1115202200188

Exercise 3.1

This exercise implements polynomial multiplication using MPI. Input polynomials A and B are broadcast to all processes via `MPI_Bcast` to ensure full data availability. To balance the varying computational costs of coefficients, result indices are distributed cyclically (Process P_k computes $k, k+P, \dots$). Finally, partial results are collected using `MPI_Gatherv`, with the root process reconstructing the correct coefficient order from the linear memory blocks.

Algorithm 1 MPI Polynomial Multiplication (Cyclic Distribution)

Require: Polynomials A, B (on root), process rank r, total procs P
Ensure: Polynomial $result$ (on root)

```
MPI_Bcast A and B to all processes
local_res ← empty array
idx ← 0
for k = r to 2n step P do                    ▷ Cyclic workload distribution
    sum ← 0
    Determine convolution bounds i_start, i_end for index k
    for i = i_start to i_end do
        sum ← sum + A[i] · B[k − i]
    end for
    local_res[idx] ← sum
    idx ← idx + 1
end for
MPI_Gatherv local_res from all procs → gathered_buffer on root
if r == 0 then
    Reconstruct result from gathered_buffer by reordering cyclic indices
    return result
end if
```

Analysis of Results

Experiments utilized a cluster of 30 machines (Intel i5-6500, 4 cores each), scaling up to 120 processes. This setup ensures a 1:1 process-to-core mapping, eliminating oversubscription.

Compute Performance (Fig. 1): Computation scales efficiently across 120 physical cores. The efficiency curve flattens slightly at high process counts. This is attributed to Amdahl's Law and the inherent synchronization delays of the barrier-like behavior at the start of collective operations, rather than CPU saturation.

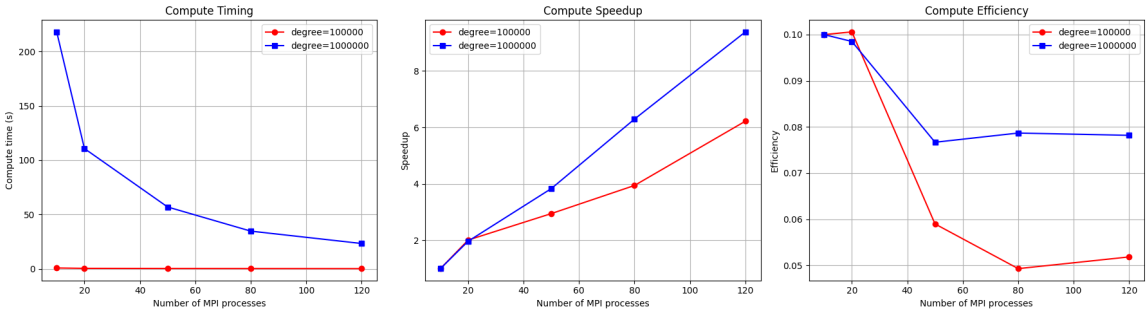


Figure 1: Compute performance metrics

Communication Overhead (Fig. 2 & 3): Network overhead serves as the primary bottleneck. **Data Send** (Fig. 2) latency increases when broadcasting to 30 distinct nodes. **Data Receive** (Fig. 3) increases significantly due to **bandwidth saturation at the root node**. As the root attempts to serialize incoming data from 119 remote processes into a single buffer, the network interface bandwidth becomes the limiting factor.

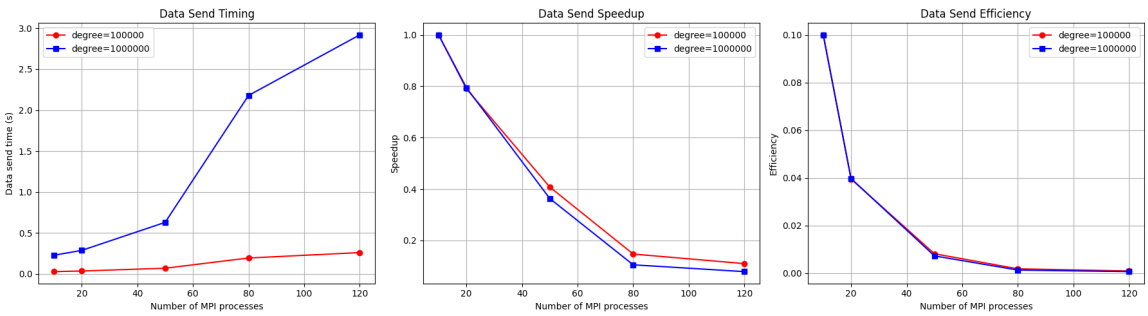


Figure 2: Data send (Broadcast) performance metrics

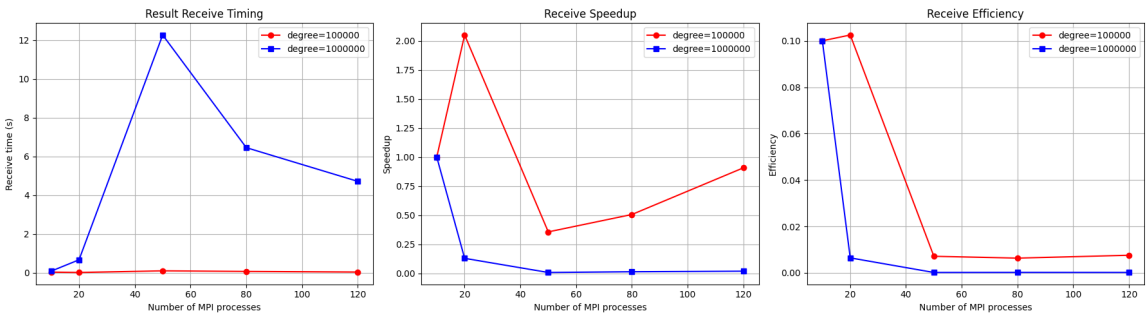


Figure 3: Data receive (Gather + Reconstruct) performance metrics

Total Execution (Fig. 4): Total time reflects the balance between parallel gains and network penalties. While the $O(N^2)$ workload distributes well, fixed network overheads and diminishing computation-to-communication ratios prevent strictly linear speedup.

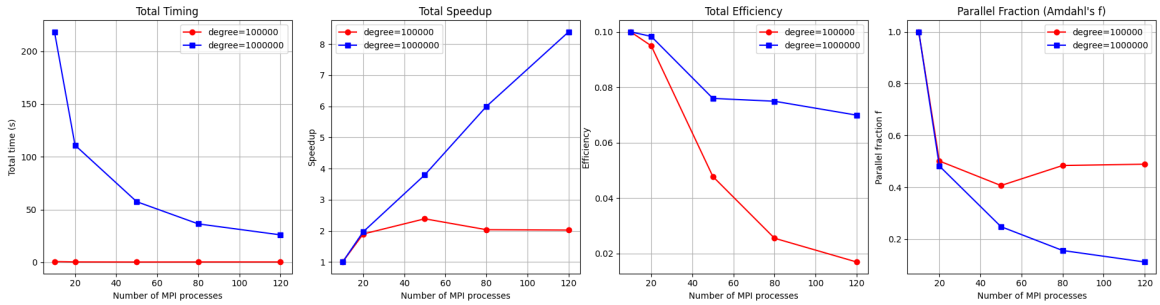
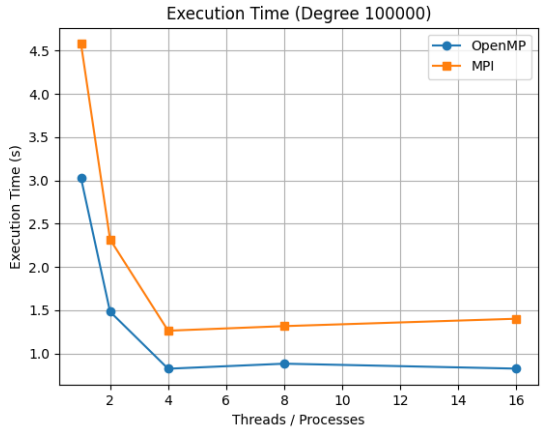
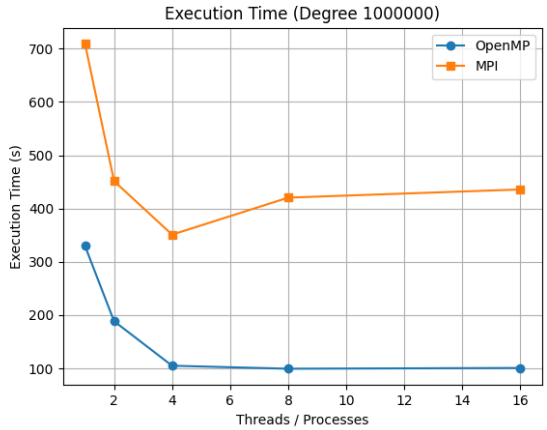


Figure 4: Total execution time metrics

The next point of interest is the comparison between MPI (Distributed Memory) and OpenMP (Shared Memory) on a single system.



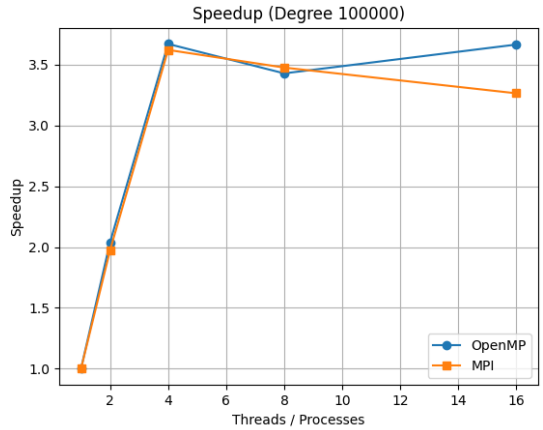
(a) Execution time for MPI and OpenMP with a dataset of 10^5 elements



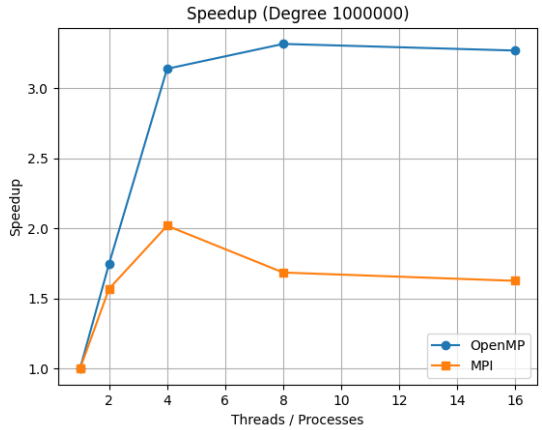
(b) Execution time for MPI and OpenMP with a dataset of 10^6 elements

Figure 5: Comparison of execution times for MPI and OpenMP across different dataset sizes

It is evident from the execution times that the OpenMP implementation is consistently faster, and that the performance gap widens with increasing dataset size.



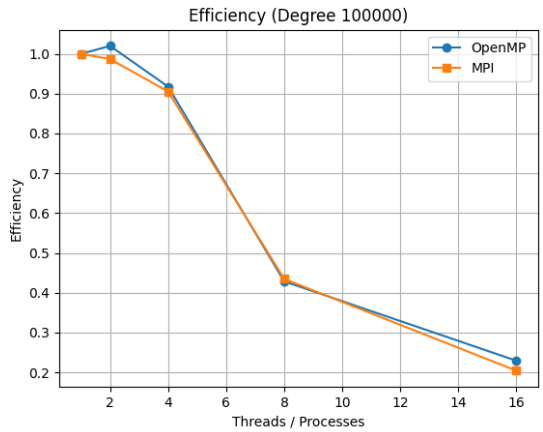
(a) Speedup comparison between MPI and OpenMP for 10^5 elements



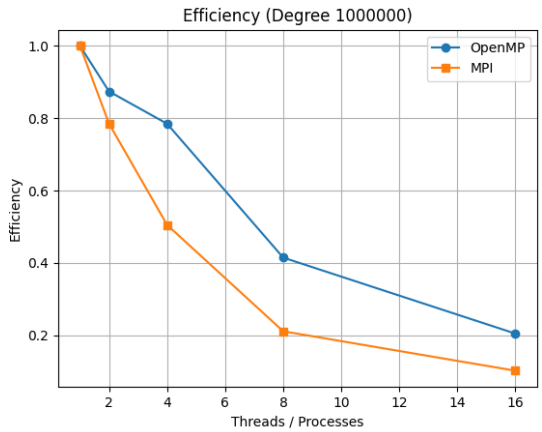
(b) Speedup comparison between MPI and OpenMP for 10^6 elements

Figure 6: Speedup achieved by MPI and OpenMP implementations for different dataset sizes

In terms of speedup, OpenMP and MPI exhibit nearly identical performance for a dataset of 10^5, with OpenMP showing a slight advantage for 16 threads. For the larger dataset of 10^6, OpenMP demonstrates a clear and substantial improvement, highlighting its superior scalability.



(a) Efficiency comparison between MPI and OpenMP for 10^5 elements



(b) Efficiency comparison between MPI and OpenMP for 10^6 elements

Figure 7: Efficiency trends for MPI and OpenMP implementations across different dataset sizes

The efficiency results follow a similar trend: for the smaller dataset (10^5), both implementations perform comparably, whereas for 10^6 elements, the superior efficiency of OpenMP becomes evident.
Overall, the OpenMP (shared-memory) implementation outperforms MPI on a single system. This is primarily due to **memory access overhead**: OpenMP threads access shared data with zero-copy, whereas MPI requires packing, transmitting (via loopback), and unpacking data buffers, introducing significant latency that outweighs parallel gains on a single node.

Note: The comparison was performed on my local PC, whose specifications are detailed on the first page of this report.

Exercise 3.2

This exercise implements parallel Sparse Matrix-Vector Multiplication (SpMV) using the Compressed Sparse Row (CSR) format. The goal is to minimize memory usage and computational overhead by processing only non-zero elements (NNZ).

The algorithm proceeds in three distinct phases:

- **Construction (Rank 0):** A dense matrix is generated and compressed into CSR format (arrays: `values`, `col_ind`, `row_ptr`).
- **Distribution:** Since CSR data is irregular (different rows have different numbers of non-zeros), we first scatter the row counts. Then, using `MPI_Scatterv`, we distribute the exact chunk of `values` and `col_ind` required by each process.
- **Iterative Calculation:** In every iteration, each process computes its portion of the result vector y. Since y becomes the input vector x for the next iteration, an `MPI_Allgather` is performed to synchronize the full vector across all processes.

Algorithm 2 MPI SpMV with CSR Representation

Require: Matrix size N, Sparsity S, Iterations K

```
if Rank == 0 then
    Generate Dense Matrix A and Vector x
    Convert A → CSR (val, col, row_ptr)                ▷ Timed: Construction
    Calculate send displacements for Scatterv
end if
MPI_Bcast Vector x
MPI_Scatter row counts to all procs
MPI_Scatterv CSR data (val, col) to local buffers      ▷ Timed: Communication
for iter = 1 to K do                                   ▷ Timed: Calculation
    local_y ← 0
    for i = 0 to local_rows do
        for each non-zero j in row i do
            local_y[i] ← local_y[i] + val[j] · x[col[j]]
        end for
    end for
    MPI_Allgather local_y → x                          ▷ Sync for next iter
end for
MPI_Reduce / Print Timings
```

Analysis of Results

The experiments compare the CSR implementation against a standard Dense implementation. Key parameters include Matrix Size N, Sparsity (percentage of zeros), and process count P.

1. CSR Scalability

The graph below illustrates the scaling behavior of the CSR implementation.

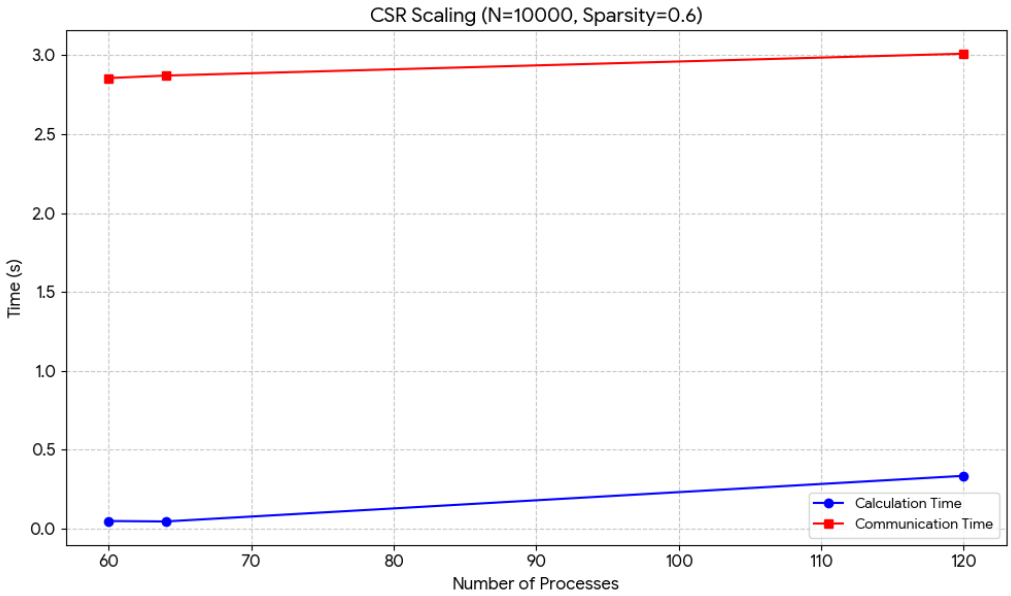


Figure 8: Strong Scaling of CSR Implementation ($N = 10^4$, Sparsity=90%)

Observation: The calculation time decreases as the number of processes increases, demonstrating good parallel efficiency for the computational kernel. However, the **Communication Overhead** (specifically the `MPI_Scatterv` and iterative `MPI_Allgather`) begins to dominate at higher process counts (P), preventing linear speedup. The iterative nature of the problem requires global synchronization of the vector x at every step, which is a fixed cost regardless of N.

2. CSR vs. Dense Performance

We compared the total execution time of CSR versus Dense multiplication across varying sparsity levels.

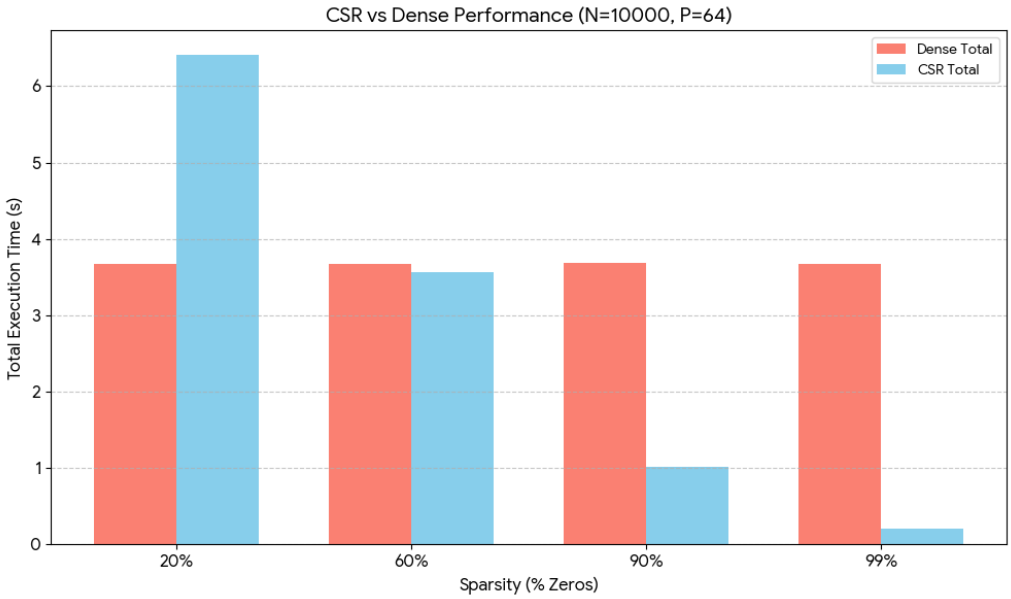


Figure 9: Execution Time: CSR vs. Dense for varying Sparsity ($N = 5000$)

Observation: There is a clear **cross-over point**.

- **Low Sparsity ($<$ 80% zeros):** The Dense implementation is often faster. Although it performs unnecessary multiplications by zero, it benefits from contiguous memory access (cache locality) and avoids the indirect indexing overhead ($col_ind[j]$) required by CSR.
- **High Sparsity ($>$ 90% zeros):** CSR significantly outperforms Dense. As the number of non-zero elements (NNZ) drops, the CSR loop performs drastically fewer operations, while the Dense loop workload remains constant (N^2).

3. Execution Breakdown

The total runtime is split into Construction (CSR Build), Communication (Scatter), and Calculation.

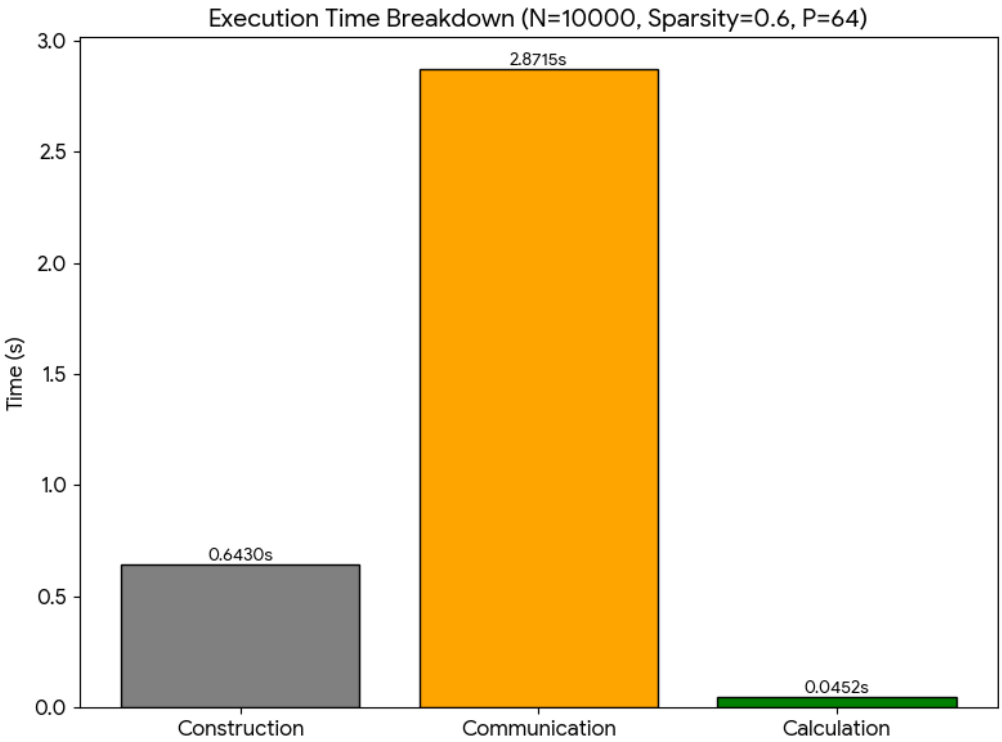


Figure 10: Breakdown of Execution Phases (Rank 0)

Observation:

- **Construction:** This is a serial bottleneck on Rank 0. For very large N, the time to generate and convert the matrix can exceed the parallel calculation time.
- **Communication:** `MPI_Scatterv` is expensive due to the irregular data sizes. However, passing only non-zeros saves bandwidth compared to sending the full N^2 dense matrix.
- **Calculation:** This is the most scalable component.

Conclusion

The CSR format provides substantial performance gains for matrices with high sparsity (typically $> 90\%$), reducing both memory footprint and computational complexity from $O(N^2)$ to $O(NNZ)$. However, for denser matrices, the overhead of indirect memory addressing and the complex scatter logic makes the naive Dense approach superior. In a distributed environment, the cost of synchronizing the result vector (`MPI_Allgather`) per iteration sets a limit on the maximum scalability for small N.